
Industry Problem Solving Task — CO4 Assessment Tool 2

Five Deep-Learning Industry Case Studies

Course: Deep Learning (MLA04) | Course Outcome: CO4 | Total Marks: 25

This document answers all five industry problem-solving questions from the CO4 Assessment Tool 2 paper. Each answer follows a consistent, step-by-step structure — problem framing, design/architecture with justification, an architecture diagram, and an illustrative code snippet — so every marking-rubric criterion (Understanding of the Problem, Application of Engineering Concepts, Solution Design, Use of Tools, Analysis) is addressed explicitly.

Q1. Transfer Learning — Skin Cancer Detection

A healthcare startup aims to develop an AI-based system for detecting skin cancer from dermoscopic images. Since only a limited number of labeled images are available, design a transfer learning strategy using a pre-trained CNN model. Justify your choice of model, identify which layers should be frozen or fine-tuned, and explain how your design improves classification performance.

Step 1 — Understanding the industry constraint

The core constraint is **data scarcity**: dermoscopic images are expensive to collect and must be labelled by dermatologists, so only a small dataset (often a few thousand images at most, e.g. datasets similar to ISIC) is available. Training a deep CNN from scratch on this little data would overfit badly. **Transfer learning** — reusing a network already trained on a very large, general image dataset (ImageNet, ~1.2 million images) — solves this by reusing already-learned low- and mid-level visual features (edges, textures, shapes, colour blobs) that are broadly useful for any image-classification task, including lesion images.

Step 2 — Choice of pre-trained model (with justification)

Recommended model: EfficientNet-B0 (alternatively ResNet-50 or DenseNet-121 as strong second choices). Justification:

- **Strong ImageNet accuracy per parameter:** EfficientNet's compound scaling of depth, width and resolution gives it excellent accuracy while remaining lightweight enough to fine-tune on a modest GPU — important for a startup with limited compute budget.
- **Proven track record in medical-imaging transfer learning:** ResNet, DenseNet, and EfficientNet backbones are the most widely used and benchmarked choices in skin-lesion classification literature (e.g., ISIC challenge leaderboards), which lowers project risk.
- **Residual/dense connections aid fine-tuning stability:** skip connections keep gradients well-behaved when only a subset of layers is unfrozen, which matters for the small-data fine-tuning regime described below.
- **Available pretrained weights and mature tooling** in Keras/PyTorch, enabling rapid, reliable implementation within the startup's development timeline.

Step 3 — Which layers to freeze vs. fine-tune

- **Freeze the early convolutional blocks** (the first ~60–70% of the network, e.g. stem + first few blocks of EfficientNet-B0 / conv blocks 1–3 of ResNet-50): these layers encode generic, low-level features — edges,

colour gradients, simple textures — that are essentially identical whether the input is a photo of a cat or a skin lesion, so retraining them wastes the scarce labelled data and risks catastrophic forgetting of useful ImageNet knowledge.

- **Unfreeze and fine-tune the later convolutional blocks** (the last 1–2 blocks, e.g. block 4–5 of ResNet-50 / the top MBConv blocks of EfficientNet-B0): these layers encode higher-level, task-specific patterns (object parts, complex shapes) and need to adapt to lesion-specific structures such as border irregularity, colour variation, and asymmetry that ImageNet classes never contained.
- **Replace the original ImageNet classification head** with a new head: Global Average Pooling → Dropout → Dense(128, ReLU) → Dense(1, Sigmoid) for binary benign/malignant classification (or Softmax over multiple lesion classes) — this head is always fully trainable from scratch, since ImageNet's 1000-class head is irrelevant to this task.

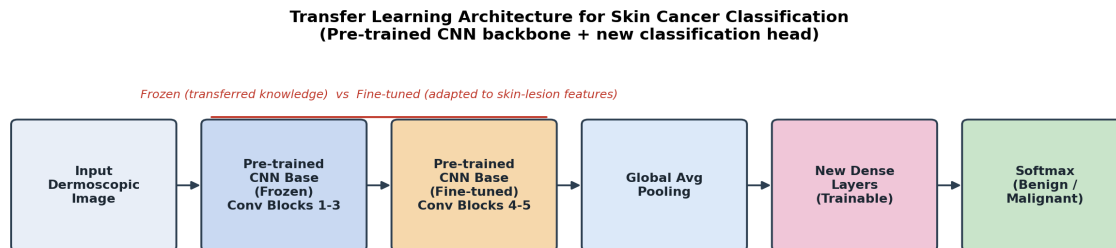


Figure Q1. Transfer-learning architecture: frozen early layers preserve general visual features, fine-tuned later layers and a new dense head adapt to skin-lesion classification.

Step 4 — Training strategy that improves classification performance

- **Two-phase training:** Phase 1 — train only the new head with the whole backbone frozen (fast convergence, prevents large random-initialised gradients from destroying pretrained weights). Phase 2 — unfreeze the last blocks and continue training with a much smaller learning rate (e.g. $1e-5$ vs $1e-3$) so the pretrained weights are adjusted gently.
- **Aggressive data augmentation** (rotation, flipping, zoom, colour jitter, hair/artifact simulation) to further combat the small-dataset problem.
- **Class-imbalance handling** (class weights or focal loss), since malignant cases are typically far rarer than benign ones in dermoscopic datasets.
- **Early stopping and dropout** on the new head to control overfitting given the fine-tuning still occurs on a small dataset.

Why this improves performance: the network starts from weights that already understand general visual structure, so it needs far fewer labelled examples to reach a given accuracy than training from scratch; freezing early layers prevents overfitting on the small dataset while fine-tuning later layers lets the model specialise in lesion-specific cues; the result is faster convergence, higher achievable accuracy, and much better generalisation to unseen patients' images than a from-scratch CNN could achieve with the same amount of labelled data.

Illustrative Code (Keras/TensorFlow)

```

import tensorflow as tf
from tensorflow.keras import layers, models, applications

IMG_SIZE = (224, 224)

# 1. Load pre-trained EfficientNetB0 (ImageNet weights), drop original head
base_model = applications.EfficientNetB0(
    include_top=False, weights="imagenet",
    input_shape=IMG_SIZE + (3,), pooling="avg"
)

# 2. Freeze all layers first (Phase 1)
base_model.trainable = False

inputs = layers.Input(shape=IMG_SIZE + (3,))
x = applications.efficientnet.preprocess_input(inputs)
x = base_model(x, training=False)
x = layers.Dropout(0.3)(x)
x = layers.Dense(128, activation="relu")(x)
outputs = layers.Dense(1, activation="sigmoid")(x) # benign(0) / malignant(1)
model = models.Model(inputs, outputs)

model.compile(optimizer=tf.keras.optimizers.Adam(1e-3),
    loss="binary_crossentropy", metrics=["accuracy", "AUC"])
# ---- Phase 1: train only the new head ----
model.fit(train_ds, validation_data=val_ds, epochs=10)

# 3. Phase 2: unfreeze last blocks for fine-tuning
base_model.trainable = True
for layer in base_model.layers[:-30]: # keep early ~70% frozen
    layer.trainable = False

model.compile(optimizer=tf.keras.optimizers.Adam(1e-5), # much smaller LR
    loss="binary_crossentropy", metrics=["accuracy", "AUC"])
model.fit(train_ds, validation_data=val_ds, epochs=15)

```

Q2. DenseNet + PixelNet — Pixel-Level Road-Scene Analysis

A smart city project requires an automated road-scene analysis system that accurately identifies roads, pedestrians, vehicles, and traffic signs at the pixel level. Design a deep learning architecture using DenseNet and PixelNet to solve this problem. Explain how the proposed architecture improves feature reuse, segmentation accuracy, and computational efficiency.

Step 1 — Understanding the problem

Identifying objects "at the pixel level" is a **semantic segmentation** task: every pixel in the input frame must be assigned a class label (road, pedestrian, vehicle, traffic sign, background/other). This is harder than whole-image classification because the network must preserve precise spatial detail (object boundaries) while still understanding high-level context (what an object is).

Step 2 — Proposed architecture: DenseNet encoder + PixelNet-style decoder

The design follows an **encoder–decoder** pattern, similar in spirit to fully-convolutional DenseNets (e.g., "The One Hundred Layers Tiramisu"), combined with PixelNet's idea of predicting each pixel's class from a feature representation sampled across multiple network depths:

- **Encoder = DenseNet:** a stack of dense blocks (with transition/down-sampling layers between them) extracts a hierarchy of features from low-level edges to high-level semantic concepts, progressively reducing spatial resolution while increasing channel depth.
- **Decoder = PixelNet-style pixel head:** instead of only using the final, coarse feature map, the decoder up-samples the encoder's feature maps back toward the input resolution and, following PixelNet's hypercolumn idea, combines features sampled from *multiple* depths (shallow layers for fine boundaries, deep layers for semantic identity) through skip connections, then applies a small MLP / 1x1-convolution classifier at every pixel location to output a per-pixel class score.
- **Output layer:** a pixel-wise softmax over the target classes (road, pedestrian, vehicle, traffic sign, background), producing a full-resolution segmentation map.

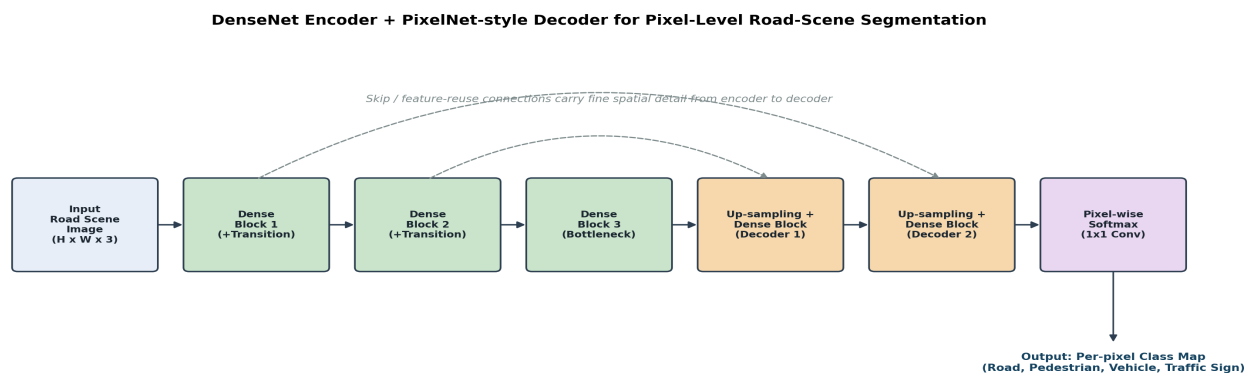


Figure Q2. DenseNet dense blocks act as the feature-extracting encoder; an up-sampling decoder with skip connections (the PixelNet-style hypercolumn idea) restores pixel-level resolution for the final per-pixel classification.

Step 3 — How the design improves feature reuse

Inside every DenseNet dense block, each layer receives the concatenated feature maps of all previous layers, so low-level edge/texture information computed early is never discarded — it remains directly available to every later layer and, via the skip connections, to the decoder as well. This means the network does not need to relearn basic

visual patterns (lane markings, vehicle silhouettes) at every depth; it reuses them, which both densifies the feature representation available for segmentation and reduces redundant learning.

Step 4 — How the design improves segmentation accuracy

- The multi-depth hypercolumn/skip-connection strategy gives the decoder access to fine spatial detail from shallow layers (needed for sharp object *boundaries*, e.g. the edge of a pedestrian) together with strong semantic context from deep layers (needed to correctly *classify* the object), which is exactly what is required for accurate pixel-level labelling in cluttered road scenes.
- Dense connectivity's strong gradient flow (Section-2-style vanishing-gradient mitigation) allows the encoder to be trained deep and stable, improving the quality of the learned features that the decoder depends on.

Step 5 — How the design improves computational efficiency

- Because dense blocks reuse features instead of relearning them, each layer only needs a small number of new feature maps (growth rate k), keeping the encoder's parameter count low compared with a plain very-deep CNN of similar depth.
- Bottleneck (1x1 conv) layers before each 3x3 convolution, and compression in the transition layers, further cut computation and memory, which matters for a smart-city deployment that may need to run inference on edge hardware mounted on traffic cameras.

Illustrative Code (PyTorch sketch)

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class DenseSegHead(nn.Module):
    """PixelNet-style decoder: up-samples and fuses multi-depth (hypercolumn)
    features from a DenseNet encoder, then classifies every pixel."""
    def __init__(self, enc_channels, num_classes):
        super().__init__()
        # 1x1 convs to project each encoder stage to a common channel size
        self.proj = nn.ModuleList([
            nn.Conv2d(c, 128, kernel_size=1) for c in enc_channels
        ])
        self.classifier = nn.Sequential(
            nn.Conv2d(128 * len(enc_channels), 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128), nn.ReLU(inplace=True),
            nn.Conv2d(128, num_classes, kernel_size=1), # per-pixel class scores
        )

    def forward(self, encoder_features, out_size):
        # encoder_features: list of feature maps from different DenseNet depths
        fused = []
        for feat, proj in zip(encoder_features, self.proj):
            f = proj(feat)
            f = F.interpolate(f, size=out_size, mode="bilinear", align_corners=False)
            fused.append(f)
        x = torch.cat(fused, dim=1) # hypercolumn: concat multi-depth features
        return self.classifier(x) # -> (B, num_classes, H, W)

# Usage: DenseNet backbone returns feature maps from 3 stages, e.g. channels [128, 256, 512]
# seg_head = DenseSegHead(enc_channels=[128, 256, 512], num_classes=5)
# logits = seg_head(encoder_features, out_size=(H, W))
# pixel_map = torch.argmax(logits, dim=1) # road / pedestrian / vehicle / sign / background
```

Q3. Bidirectional RNN + Encoder–Decoder — Multilingual Chatbot

A multinational customer service company wants to build a multilingual chatbot capable of translating customer queries while preserving contextual meaning. Design an Encoder–Decoder sequence-to-sequence model using Bidirectional RNNs. Illustrate the architecture and justify how bidirectional processing improves translation accuracy.

Step 1 — Understanding the problem

Machine translation is a **sequence-to-sequence (Seq2Seq)** problem: a variable-length input sentence in one language must be mapped to a variable-length output sentence in another language, while preserving meaning, word order differences between languages, and long-range grammatical dependencies (e.g., subject–verb agreement, pronoun references).

Step 2 — Proposed architecture

- **Encoder — Bidirectional RNN (BiLSTM/BiGRU):** the input query is processed by two RNNs, one reading the sentence left-to-right and one reading it right-to-left. At every input word, the forward and backward hidden states are concatenated, so each word's representation carries context from *both* the words before and after it.
- **Context formation:** the final forward and backward hidden states (or, in an attention-augmented version, all concatenated per-word states) are combined into a context representation that summarises the meaning of the entire input query.
- **Decoder — unidirectional LSTM:** generates the translated sentence one token at a time, conditioned on the context vector and on the tokens it has generated so far (a decoder must stay unidirectional/causal, since it cannot see future output words it hasn't generated yet).

Bidirectional-RNN Encoder + LSTM Decoder (Seq2Seq) for Multilingual Chatbot Translation

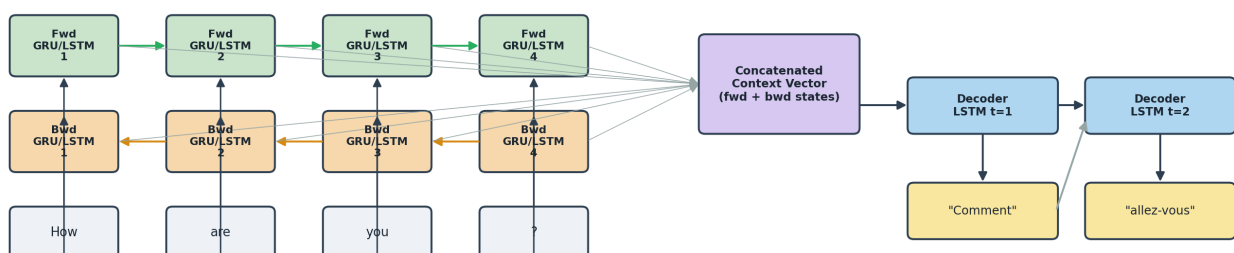


Figure Q3. Bidirectional RNN encoder reads the query in both directions and concatenates forward/backward states into a context vector; a unidirectional LSTM decoder then generates the translated output token by token.

Step 3 — Why bidirectional processing improves translation accuracy

- **Full sentence context per word:** a unidirectional encoder only knows what came before a given word; a bidirectional encoder also knows what comes after it, which is essential for resolving ambiguity — e.g., the correct sense of a word or the correct pronoun antecedent often depends on later words in the sentence.

- **Handles word-order differences between languages:** different languages order subject, verb, and object differently; a bidirectional encoding gives the decoder a richer, order-independent summary of the sentence's meaning, rather than an encoding biased toward only the source language's word order.
- **Improves long-range dependency capture:** combined with LSTM/GRU gating, the backward pass gives an additional path for information (and gradients) to travel, reducing the chance that important context from the end of a long customer query is diluted by the time the network reaches the end of the forward pass alone.
- **Better disambiguation of context-dependent meaning** — critical for customer-service queries, where preserving the customer's precise intent (not just literal words) directly affects service quality.

Illustrative Code (PyTorch Seq2Seq skeleton)

```
import torch
import torch.nn as nn

class BiRNNEncoder(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.rnn = nn.LSTM(embed_dim, hidden_dim, batch_first=True,
                           bidirectional=True)
        # project concatenated fwd+bwd states back to decoder hidden size
        self.fc_h = nn.Linear(hidden_dim * 2, hidden_dim)
        self.fc_c = nn.Linear(hidden_dim * 2, hidden_dim)

    def forward(self, src):
        embedded = self.embedding(src) # (B, T, E)
        outputs, (h, c) = self.rnn(embedded) # h: (2, B, H)
        h_cat = torch.cat([h[0], h[1]], dim=1) # fwd + bwd
        c_cat = torch.cat([c[0], c[1]], dim=1)
        hidden = torch.tanh(self.fc_h(h_cat)).unsqueeze(0)
        cell = torch.tanh(self.fc_c(c_cat)).unsqueeze(0)
        return outputs, (hidden, cell) # outputs used for attention

class LSTMDecoder(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.rnn = nn.LSTM(embed_dim, hidden_dim, batch_first=True)
        self.fc_out = nn.Linear(hidden_dim, vocab_size)

    def forward(self, input_token, hidden_cell):
        embedded = self.embedding(input_token) # (B, 1, E)
        output, hidden_cell = self.rnn(embedded, hidden_cell)
        logits = self.fc_out(output.squeeze(1)) # (B, vocab_size)
        return logits, hidden_cell

# encoder = BiRNNEncoder(src_vocab, 256, 512)
# decoder = LSTMDecoder(tgt_vocab, 256, 512)
# enc_outputs, (h, c) = encoder(src_batch)
# logits, (h, c) = decoder(prev_token, (h, c)) # repeated per output time step
```

Q4. BPTT + LSTM — Stock Price Forecasting

A financial analytics company is developing a model to forecast stock prices using historical market data. Create an LSTM-based prediction system and explain how Backpropagation Through Time (BPTT) is used to train the network. Justify why LSTM is more suitable than a traditional RNN for this application.

Step 1 — Understanding the problem

Stock-price forecasting is a **time-series regression** problem: given a window of past observations (price, volume, technical indicators, etc.), predict future price(s). The data has both short-term fluctuations and longer-term trend/momentum patterns, so the model must be able to remember relevant information over many time steps.

Step 2 — Proposed LSTM-based prediction system

- **Input representation:** a sliding window of the last N trading days' features (e.g. open, high, low, close, volume, moving averages), normalised (e.g. min-max or z-score scaling).
- **Model:** one or more stacked LSTM layers process the input sequence step by step, maintaining a hidden state h and cell state c that summarise relevant history.
- **Output head:** a Dense layer on the final LSTM hidden state predicts the next day's (or next several days') closing price (regression, typically with a linear activation).

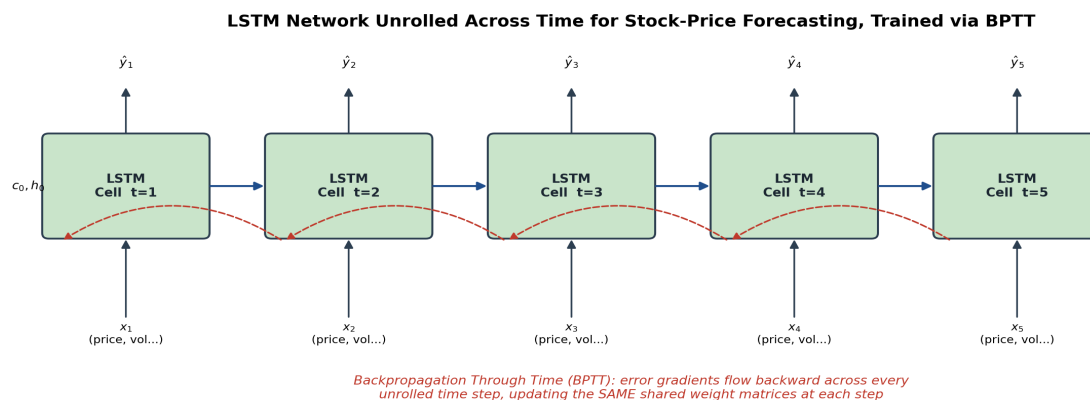


Figure Q4. The LSTM is unrolled across time steps for training; Backpropagation Through Time (BPTT) propagates the error gradient backward across every unrolled step to update the shared weight matrices.

Step 3 — How BPTT trains the network

- **Unrolling:** conceptually, the LSTM is "unrolled" into a chain with one copy of the cell per time step (Figure Q4), all copies sharing the same weight matrices.
- **Forward pass:** the sequence is fed through the unrolled chain, producing a prediction (and loss against the true next-day price) at each relevant time step.
- **Backward pass:** starting from the loss at the final (or every) time step, gradients are propagated backward through each unrolled cell using the chain rule, all the way back to the first time step — this is Backpropagation Through Time.
- **Gradient accumulation and weight update:** because every unrolled copy shares the same weights, the gradients computed at every time step are summed for each shared weight matrix before a single update step (e.g. via Adam) is applied.

- **Truncated BPTT:** for long historical sequences, gradients are typically propagated back only a fixed number of steps (a truncation window) rather than through the entire history, to keep training computationally tractable and numerically stable.

Step 4 — Why LSTM is more suitable than a traditional RNN here

- **Vanishing/exploding gradients:** in a plain RNN, BPTT gradients are repeatedly multiplied by the same recurrent weight matrix at every time step; over long price histories this product shrinks toward zero (or explodes), so the network effectively "forgets" information from many steps ago — exactly the long-range trend information that matters for forecasting.
- **Gated memory cell:** LSTM's forget, input, and output gates let the network learn *what to keep, update, or discard* in its cell state at each step, giving it an additive (rather than purely multiplicative) path for gradients, which keeps gradients much more stable across long sequences during BPTT.
- **Better long-term dependency modelling:** stock movements are influenced by both recent volatility and longer-term momentum/trend; LSTM's ability to preserve information over many time steps makes it far better suited than a vanilla RNN to capture both.

Illustrative Code (Keras LSTM forecaster)

```
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models

def make_sequences(prices, window=30):
    X, y = [], []
    for i in range(len(prices) - window):
        X.append(prices[i:i+window])
        y.append(prices[i+window])
    return np.array(X), np.array(y)

# prices: normalised 1-D array of historical closing prices
X, y = make_sequences(prices, window=30)
X = X.reshape((X.shape[0], X.shape[1], 1)) # (samples, time_steps, features)

model = models.Sequential([
    layers.LSTM(64, return_sequences=True, input_shape=(30, 1)),
    layers.LSTM(32),
    layers.Dense(16, activation="relu"),
    layers.Dense(1) # predicted next-day price
])

model.compile(optimizer="adam", loss="mse", metrics=["mae"])
# Training runs forward pass -> loss -> BPTT backward pass -> weight update each batch
model.fit(X, y, epochs=50, batch_size=32, validation_split=0.1)
```

Q5. Integrated Deep Learning Solution — Automatic Video Captioning

A media streaming company plans to automatically generate captions for uploaded videos to improve accessibility. Design an integrated deep learning solution that combines Transfer Learning for visual feature extraction with an LSTM-based Encoder–Decoder architecture for caption generation. Explain the role of each component and justify how the complete system produces accurate captions.

Step 1 — Understanding the problem

Automatic video captioning requires two very different kinds of understanding at once: **visual** understanding of what is happening in the video frames, and **language** understanding/generation to describe it as a grammatical sentence. This naturally motivates combining a visual feature extractor with a sequence-generation model — exactly the two techniques named in the question.

Step 2 — Integrated architecture and the role of each component

- **1. Frame sampling:** a set of representative frames is extracted from each uploaded video at a fixed rate (e.g., 1–4 frames per second).
- **2. Transfer-learned CNN (visual feature extractor):** a CNN pretrained on ImageNet (e.g. ResNet-50 / EfficientNet, as justified in Q1) processes each sampled frame and outputs a compact feature vector summarising its visual content. *Role:* supplies rich, general-purpose visual semantics (objects, scenes, actions) without needing millions of video-specific training examples — the same data-efficiency argument as Q1.
- **3. LSTM Encoder (temporal aggregation):** the sequence of per-frame feature vectors is fed into an LSTM that processes them in temporal order, building a single context representation that captures how the scene changes over time (motion, actions, transitions). *Role:* converts a variable-length sequence of frame features into a fixed-size video-level summary.
- **4. Context Vector:** the encoder's final hidden state (optionally combined with an attention mechanism over all frame features, as in Q3) forms the bridge between the visual encoder and the language decoder.
- **5. LSTM Decoder (language generation):** conditioned on the context vector, an LSTM decoder generates the caption one word at a time, similarly to the translation decoder in Q3, using previously generated words plus the video context to predict the next word.
- **6. Output:** the decoded word sequence forms the final caption, which is attached to the video for accessibility (e.g. as captions/subtitles or alt-text).

Integrated Video-Captioning Pipeline: Transfer-Learned CNN Encoder + LSTM Encoder-Decoder

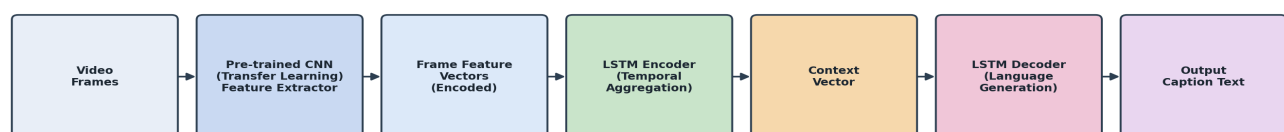


Figure Q5. End-to-end pipeline: a transfer-learned CNN extracts per-frame visual features, an LSTM encoder aggregates them over time into a context vector, and an LSTM decoder generates the caption text word by word.

Step 3 — Why the integrated system produces accurate captions

- **Transfer learning solves the visual data-efficiency problem:** as in Q1, reusing ImageNet-pretrained convolutional features means the system does not need a huge, purpose-labelled video dataset to recognise objects and scenes reliably — it starts from strong, general visual understanding.
- **The LSTM encoder captures temporal/action information** that a single-frame classifier would miss entirely — many captions describe an *action* ("a person is running") that only makes sense across multiple frames.
- **The LSTM decoder ensures grammatically fluent, contextually appropriate language** by conditioning each generated word on both the video context and everything generated so far, exactly as in Q3/Q4's justification for LSTM's gated memory over long sequences.
- **Modularity:** because each component (visual encoder, temporal encoder, language decoder) is trained for its own well-understood sub-problem, the combined system generalises better and is easier to debug/improve than one monolithic network trained end-to-end from raw pixels to text with limited data.

Illustrative Code (PyTorch, CNN + LSTM Encoder–Decoder captioning)

```
import torch
import torch.nn as nn
import torchvision.models as tv_models

class CNNFeatureExtractor(nn.Module):
    """Transfer-learning visual encoder (Q1-style, frozen/fine-tuned ResNet-50)."""
    def __init__(self, feature_dim=512):
        super().__init__()
        resnet = tv_models.resnet50(weights="IMAGENET1K_V2")
        modules = list(resnet.children())[:-1] # drop original FC head
        self.backbone = nn.Sequential(*modules)
        for p in list(self.backbone.parameters())[:-20]: # freeze early layers
            p.requires_grad = False
        self.proj = nn.Linear(resnet.fc.in_features, feature_dim)

    def forward(self, frames): # frames: (B, T, 3, 224, 224)
        B, T = frames.shape[:2]
        x = frames.view(B * T, *frames.shape[2:])
        feats = self.backbone(x).flatten(1) # (B*T, 2048)
        feats = self.proj(feats).view(B, T, -1) # (B, T, feature_dim)
        return feats

class VideoCaptioner(nn.Module):
    def __init__(self, feature_dim, vocab_size, embed_dim, hidden_dim):
        super().__init__()
        self.cnn = CNNFeatureExtractor(feature_dim)
        self.temporal_encoder = nn.LSTM(feature_dim, hidden_dim, batch_first=True)
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.decoder = nn.LSTM(embed_dim, hidden_dim, batch_first=True)
        self.fc_out = nn.Linear(hidden_dim, vocab_size)

    def forward(self, frames, caption_in):
        frame_feats = self.cnn(frames) # visual features
        _, (h, c) = self.temporal_encoder(frame_feats) # context vector
        embedded = self.embedding(caption_in)
        dec_out, _ = self.decoder(embedded, (h, c)) # generate words
        return self.fc_out(dec_out) # (B, T, vocab_size)
```